

FINAL METHODOLOGY AND DOCUMENTATION

Amazon ML Challenge 2026

1. Problem Statement / Objective

The objective is to perform multi-source entity resolution across business records originating from Source 1 (S1), Source 2 (S2), and Source 3 (S3). The task is to identify which entities in S2 and S3 correspond to each entity in S1 despite differences in business names, addresses, formatting, spelling, and other record attributes.

The problem is formulated as a many-to-many entity matching task. A single S1 entity may correspond to zero, one, or multiple entities in the target sources. Therefore, the system does not assume a one-to-one correspondence between records.

The proposed pipeline combines multiple stages: candidate generation through multi-view retrieval, pairwise comparison of candidate records, cross-source graph consistency, and an entity-level multi-match decision. The retrieval stage reduces the number of target records considered by the downstream matching model, while the pairwise and graph stages provide progressively more detailed evidence for determining entity correspondence.

The overall objective is to produce, for every S1 entity, the set of corresponding S2 and S3 entity IDs in the required submission format while maintaining a scalable matching process and preserving the possibility of multiple valid matches.

2. Overall Methodology / Architecture

The proposed system is organized as a sequential entity-resolution pipeline in which each stage provides evidence or candidates for the next stage. The architecture is designed for a many-to-many matching setting and separates candidate generation from pairwise matching and final entity-level decision making.

The complete workflow is:

Source 1 Records

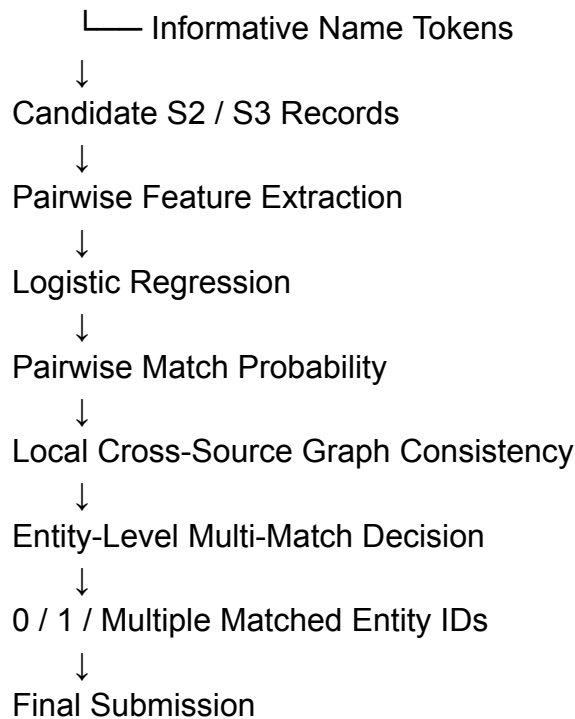


Entity Representation



Multi-View Candidate Retrieval

- |— Normalized Exact Name
- |— Normalized Exact Address



The first stage represents the business records using their available attributes, including business name, business address, country, and entity identifier. The retrieval stage then uses complementary blocking strategies to generate a manageable set of candidate S2 and S3 entities.

Each retrieved candidate is subsequently compared with its S1 record using pairwise features based on name, address, country, and missing-value information. Logistic Regression produces a pairwise match probability that serves as intermediate evidence rather than the final entity-level decision.

The resulting pairwise evidence is then used by the local cross-source graph stage to examine consistency between candidate entities across S2 and S3. Finally, the entity-level decision stage determines the set of matched entity IDs for each S1 record. Since the problem permits many-to-many correspondence, the final output may contain zero, one, or multiple matched entities.

The architecture deliberately separates retrieval, pairwise matching, graph consistency, and final decision making so that candidate generation remains computationally scalable while downstream stages can use richer evidence for entity-level matching.

3. Entity Representation

Each business record is represented using four primary attributes:

Attribute	Description
-----------	-------------

<code>entity_id</code>	Unique identifier of the business record within its source
<code>business_name</code>	Business or organization name
<code>business_address</code>	Business address
<code>country</code>	Country associated with the business

The same basic representation is used for records from S1, S2, and S3. The `entity_id` is retained to identify records and to construct the final matching output, while the business name, address, and country provide the information used during retrieval and pairwise comparison.

Business names and addresses may contain formatting differences, spelling variations, missing values, and different writing systems. Therefore, text normalization is applied before retrieval, while the pairwise matching stage uses multiple similarity measures to compare candidate records.

Missing values are explicitly preserved as part of the representation. In particular, an absent business address does not prevent a record from being considered through the name-based or informative-token retrieval views.

The source identity is also retained throughout the pipeline. Thus, candidate records are associated explicitly with their target source, S2 or S3, rather than treating all target records as belonging to a single undifferentiated dataset.

4. Candidate Generation / Blocking

4.1 Purpose of Candidate Generation

Candidate generation is the first stage of the entity-resolution pipeline and is responsible for reducing the number of target records that must be evaluated by the downstream pairwise matching model. For each Source-1 (S1) entity, the retrieval stage identifies a set of potentially relevant entities from Source-2 (S2) and Source-3 (S3). These retrieved records are treated only as candidates and are not considered confirmed matches at this stage.

Because the target sources contain millions of records, comparing every S1 entity against every S2/S3 entity would be computationally expensive. The retrieval stage therefore uses multiple

complementary blocking strategies to obtain a substantially smaller candidate set while retaining as many true matches as possible.

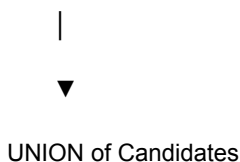
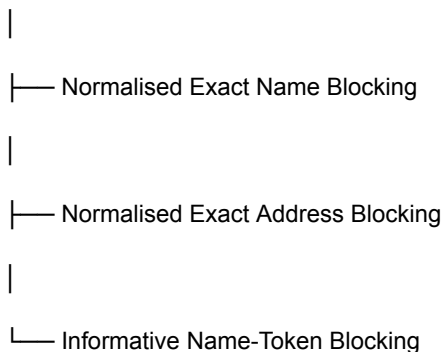
The final retrieval system uses three complementary views: normalised exact name blocking, normalised exact address blocking, and informative name-token blocking. Candidates retrieved by these views are combined using a **union operation**. The resulting candidates are then passed to the pairwise matching stage for detailed comparison.

4.2 Multi-View Retrieval Architecture

The retrieval system uses three complementary blocking views to generate candidate entities for each S1 record. The three views are applied independently, and their results are combined using a **union** operation so that a candidate retrieved by any one of the views is retained.

The final retrieval architecture is:

S1 Record



Candidate S2 / S3 Records

The three views are complementary. Exact name blocking can retrieve records with identical normalized business names, exact address blocking can recover matches where address information is shared, and informative name-token blocking provides additional candidates when the complete business names are not identical but informative name tokens overlap.

The union strategy is used rather than intersection because requiring a candidate to satisfy multiple blocking conditions could remove true matches that are identifiable through only one retrieval view.

The same retrieval architecture is applied separately to S2 and S3 using their respective target indexes. Retrieval therefore produces candidate S2 and candidate S3 records that are subsequently evaluated by the pairwise matching stage.

4.3 Text Normalisation

Text normalization is applied before constructing the name, address, and token-based retrieval indexes. The normalization procedure is designed to reduce superficial differences in representation while preserving characters from different writing systems.

The implemented normalization performs:

- Unicode **NFKC normalization**
- conversion to **lowercase**
- removal of punctuation and other non-letter/non-number symbols
- normalization of repeated whitespace
- trimming of leading and trailing whitespace

Unicode letters, numbers, and combining marks are preserved. This is important because the dataset contains businesses represented using multiple scripts, including non-Latin writing systems.

For exact name and address blocking, the normalized text is converted into a compact representation by removing whitespace. For informative name-token blocking, the normalized name is split into whitespace-separated tokens. This normalization is applied consistently to both the S1 records and the corresponding S2/S3 target records before retrieval.

4.4 Normalised Exact Name Blocking

For name-based blocking, the business name of each target record is normalized using the procedure described above and converted into a compact representation by removing whitespace. An index is then constructed using the combination of **country and normalised business name**.

For each S1 record, its normalized business name and country are used to perform an exact lookup in the corresponding target index. All target entities associated with the matching key are returned as name-based candidates.

The name blocker therefore retrieves candidates when the business names are identical after normalization, while avoiding expensive pairwise fuzzy comparisons across the complete target corpus.

Records with missing or empty business names do not generate name-based candidates. However, they can still be retrieved through the other blocking views.

4.5 Normalised Exact Address Blocking

Address-based blocking uses the same normalization procedure applied to business names. The normalized address is converted into a compact representation by removing whitespace, and an index is constructed using the combination of **country and normalized business address**.

For each S1 record, the normalized address and country are used to perform an exact lookup in the corresponding S2 or S3 target index. All target entities associated with the matching key are returned as address-based candidates.

Address blocking provides a complementary retrieval path to name blocking and can recover candidates when the business-name representations differ but the normalized addresses are identical.

Address information is not mandatory for retrieval. If an S1 record has a missing or empty address, the address blocker returns no candidates for that view, while the name and informative-token blocking views can still retrieve candidates.

4.6 Informative Name-Token Blocking

Informative name-token blocking provides a broader retrieval path for business names that are not identical after normalization. The normalized business name is tokenized, and token frequencies are calculated over the target source.

To avoid extremely common tokens generating excessively large candidate sets, only tokens with a frequency between **5 and 500 occurrences** in the target source are indexed. For each S1 business name, the eligible tokens are ordered by frequency, and the **two least frequent eligible tokens** are selected as the most informative tokens for retrieval.

The target index is constructed using the combination of **country and token**. Candidates associated with either of the selected informative tokens are retrieved and combined into the token-based candidate set.

This approach provides additional recall for cases involving partial names, spelling variations, reordered terms, and other name variations that may not satisfy normalized exact-name blocking, while limiting the candidate volume produced by very common business-name terms.

4.7 Candidate Union and Provenance

The candidate sets produced by the three retrieval views are combined using a **union operation**. A target entity is retained as a candidate if it is retrieved by at least one of the name, address, or informative-token blocking views.

Duplicate candidate entity IDs retrieved by multiple views are merged into a single candidate record. During this process, the retrieval system retains the views that produced each candidate through the `candidate_source` field.

For example, a candidate may have:

`candidate_source = name`

if it was retrieved only through name blocking, or:

`candidate_source = name|token`

if it was retrieved through both name and token blocking.

The `candidate_source` field is **retrieval provenance metadata only**. It does not represent match confidence, probability, or a prediction from the pairwise model.

Retaining provenance also allows the downstream pipeline to identify which retrieval views contributed a candidate while keeping the candidate itself as a single entity for subsequent pairwise evaluation.

4.8 S2/S3 Retrieval

The candidate sets produced by the three retrieval views are combined using a **union operation**. A target entity is retained as a candidate if it is retrieved by at least one of the name, address, or informative-token blocking views.

Duplicate candidate entity IDs retrieved by multiple views are merged into a single candidate record. During this process, the retrieval system retains the views that produced each candidate through the `candidate_source` field.

For example, a candidate may have:

`candidate_source = name`

if it was retrieved only through name blocking, or:

`candidate_source = name|token`

if it was retrieved through both name and token blocking.

The `candidate_source` field is **retrieval provenance metadata only**. It does not represent match confidence, probability, or a prediction from the pairwise model.

Retaining provenance also allows the downstream pipeline to identify which retrieval views contributed a candidate while keeping the candidate itself as a single entity for subsequent pairwise evaluation.

4.9 Retrieval Interface

The finalized retrieval implementation is exposed through two main functions. First, `build_retrieval_indexes(target_df)` constructs the name, address, and informative-token indexes for a target source. The function is called separately for S2 and S3.

For an individual S1 record, `retrieve_candidates(source1_row, indexes)` performs lookup across all three retrieval views and returns the union of the retrieved candidates.

The output for each candidate has the following structure:

```
{  "entity_id": "...",  "candidate_source": "name|address|token"}
```

Here, `entity_id` identifies the retrieved target entity, while `candidate_source` records the retrieval view or combination of views that produced the candidate. The target source, S2 or S3, is maintained explicitly by the surrounding pipeline.

The retrieval output is passed to the pairwise matching stage, where each candidate entity is resolved to its complete S2 or S3 record and compared with the corresponding S1 record. The retrieval interface therefore forms the boundary between candidate generation and pairwise matching:

S1 record



retrieve_candidates()



candidate entity_id + candidate_source



resolve S2/S3 candidate record



pairwise feature extraction

Candidates produced by this interface are **not confirmed matches**. They are records retained for downstream evaluation by the pairwise matching and entity-level decision stages.

5. Pairwise Matching / Model Architecture

Following multi-view candidate retrieval, the pairwise matching module evaluates each retrieved candidate independently to estimate the likelihood that the candidate represents the same real-world entity as the source record.

The T2 retrieval module provides, for each S1 record, a set of candidate identifiers along with their retrieval provenance:

candidate_entity_id

candidate_source

where **candidate_source** indicates the retrieval view that produced the candidate, such as **name**, **address**, or **token**. This field is retained as **retrieval provenance metadata** and is not treated as a confidence score or used as a pairwise model feature.

Each candidate identifier is resolved to its corresponding target record, with the target source explicitly maintained as either **S2 or S3**. T3 then constructs an S1–candidate pair and extracts a fixed 12-dimensional feature vector.

The pairwise matching architecture is:

S1 Record



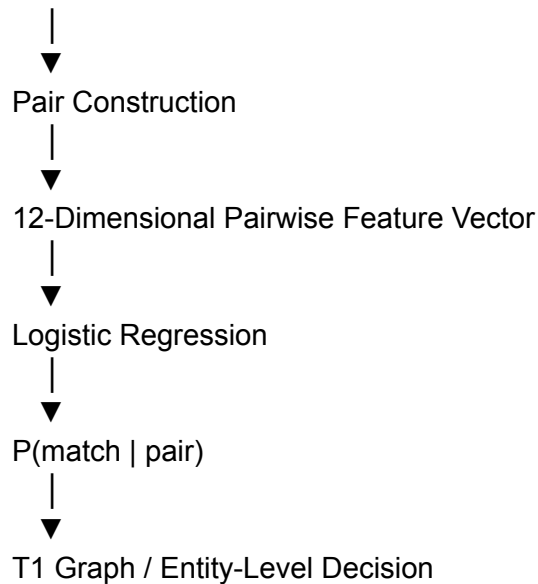
T2 Multi-View Retrieval

├── candidate_entity_id

└── candidate_source



Candidate S2/S3 Record



Pairwise Model

A **Logistic Regression classifier** is used as the pairwise matching model. The model is configured with class balancing to account for the imbalance between matching and non-matching entity pairs.

The model outputs a continuous probability:

$P(\text{match} \mid \text{pair features})$

representing the model's estimated likelihood that the two records refer to the same entity.

The pairwise probability is an **intermediate matching signal**, not the final entity-level decision. T1 subsequently uses the scored candidate relationships within the graph-consistency and multi-match decision stages.

The current implementation uses a probability threshold of **0.50 only for baseline validation**. This threshold should not be interpreted as the final challenge decision threshold.

6. Local Graph Consistency

The local graph consistency stage operates on the candidate pairs that have been evaluated by the pairwise matching stage. The purpose of this stage is to incorporate consistency between candidate entities from different target sources rather than making the final decision independently for every S1–S2 or S1–S3 pair.

For each S1 entity, the locally retrieved and pairwise-scored S2 and S3 candidates form a small entity-level graph. S1-to-target relationships are represented using the pairwise matching evidence, while relationships between S2 and S3 candidates provide additional cross-source consistency evidence.

The graph is evaluated locally for each S1 entity rather than constructing a single global graph over the entire dataset. This keeps the graph computation bounded by the retrieved candidate sets and allows the final decision stage to consider supporting and contradictory evidence before producing the matched entity IDs.

7. Entity-Level Multi-Match Decision

The entity-level decision stage converts the pairwise matching evidence and local cross-source graph evidence into the final set of matched entities for each S1 record.

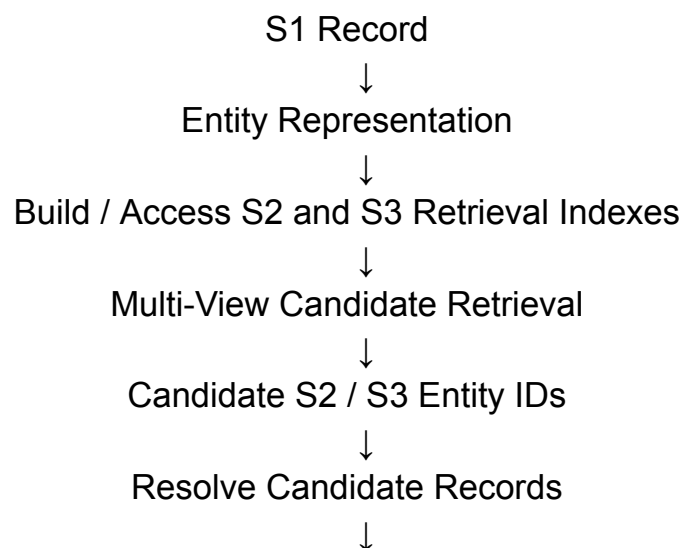
Unlike a one-to-one entity-resolution formulation, the system allows an S1 entity to have zero, one, or multiple valid matches across S2 and S3. Therefore, the final decision is made at the entity level rather than by independently selecting a single highest-scoring candidate.

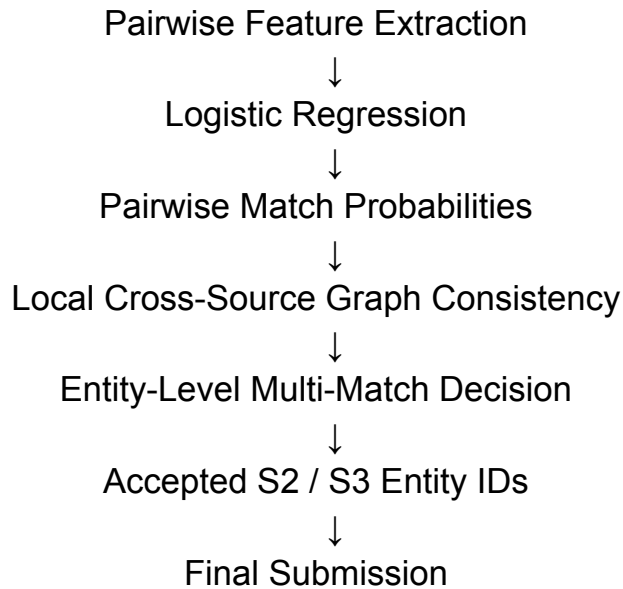
For each S1 entity, the decision stage considers the available candidate relationships and the supporting or contradictory evidence identified by the local graph stage. The resulting decision determines which S2 and S3 entity IDs are retained in the final output.

The pairwise match probability is therefore treated as intermediate evidence, rather than as the final prediction by itself. The final accepted entity IDs are produced only after the entity-level consistency and multi-match decision process.

8. End-to-End Inference Pipeline

During inference, each S1 record is processed through the complete entity-resolution pipeline. The stages are executed sequentially, with the output of each stage becoming the input to the next stage.





For each S1 record, the retrieval stage first generates candidate S2 and S3 entities using the three complementary blocking views. Each retrieved candidate is then resolved to its corresponding target record and paired with the S1 record.

The pairwise matching stage extracts the defined comparison features and produces a match probability for each candidate pair. These probabilities, together with the candidate relationships across S2 and S3, are passed to the local graph consistency stage.

The final entity-level decision stage uses this accumulated evidence to determine the matched entities for the S1 record. Because the task permits many-to-many correspondence, the resulting set can contain zero, one, or multiple S2/S3 entity IDs.

The final accepted entity IDs are then formatted according to the required submission structure.

9. Feature Engineering

The pairwise model compares the textual and structured attributes of an S1 record against each retrieved S2/S3 candidate.

A total of 12 features are generated for every candidate pair.

9.1 Business Name Features

Three complementary similarity measures are calculated:

Feature	Description
<code>name_char_similarity</code>	Character-level similarity between normalized business names
<code>name_token_similarity</code>	Jaccard similarity between the sets of name tokens
<code>name_edit_similarity</code>	Normalized edit-distance similarity between the names

These features provide complementary signals: character similarity captures spelling-level resemblance, token similarity captures shared words, and edit similarity captures overall textual closeness.

9.2 Address Features

Four features are extracted from the business addresses:

Feature	Description
<code>address_char_similarity</code>	Character-level similarity between normalized addresses
<code>address_token_similarity</code>	Jaccard similarity between address token sets
<code>address_edit_similarity</code>	Normalized edit-distance similarity
<code>address_number_overlap</code>	Jaccard overlap between numeric components extracted from the addresses

The numeric-overlap feature is particularly useful for distinguishing addresses that may share similar locality or street text but differ in house, building, postal, or other numeric components.

9.3 Country Feature

Feature	Description
<code>country_same</code>	Binary indicator showing whether both records contain the same normalized country value

Country is treated as an input attribute rather than assuming a fixed set of countries. This allows the model to operate on previously unseen country values in the test data without hard-coded country-specific logic.

9.4 Missing-Value Features

Four binary indicators explicitly represent missing information:

Feature	Description
<code>name_a_missing</code>	Whether the first record's business name is missing
<code>name_b_missing</code>	Whether the candidate record's business name is missing
<code>address_a_missing</code>	Whether the first record's address is missing
<code>address_b_missing</code>	Whether the candidate record's address is missing

Explicit missingness indicators prevent missing fields from being indistinguishable from genuinely low textual similarity.

9.5 Text Normalization

Before similarity computation, text values undergo basic normalization including:

- Unicode normalization
- lowercasing
- punctuation replacement
- whitespace normalization
- handling of empty or missing values

The same feature extraction process is applied consistently during training and inference.

Retrieval metadata is deliberately excluded from the feature vector. In particular, `candidate_source` indicates how T2 retrieved a candidate and is not interpreted as a model confidence score.

10. Validation / Experiments

The pairwise model was initially validated using a sampled training/validation experiment designed to verify the end-to-end pair construction, feature extraction, training and inference pipeline.

Training Pair Construction

Positive pairs were generated from the provided ground-truth mappings by pairing S1 records with their known S2/S3 matches. Up to three positive matches per S1 entity were used in the sampled experiment.

Negative pairs were sampled from target records that were not present in the corresponding ground-truth matches.

An important distinction is maintained between negative pairs and unretrieved pairs. Because the T2 retrieval stage does not have 100% recall, a ground-truth

match that was not retrieved by T2 must not automatically be treated as a negative training example.

The current experiment therefore uses sampled/random negatives rather than claiming that all non-retrieved or non-matched candidate relationships represent the complete negative distribution of the final system.

Model Training

The sampled pair dataset was split into training and validation subsets using a stratified 80/20 split.

The Logistic Regression model was trained with:

```
max_iter = 1000
class_weight = balanced
random_state = 42
```

Baseline Validation Results

At a probability threshold of 0.50, the current pairwise-model validation produced:

Metric	Result
Precision	0.5714
Recall	1.0000
F0.5	0.6250
Validation threshold	0.50

These results represent sampled pairwise-model validation, not final challenge performance.

The validation dataset uses sampled/random negatives and therefore does not fully reproduce the candidate distribution generated by the complete T2 retrieval pipeline. Furthermore, final challenge performance depends on downstream graph consistency and entity-level multi-match decisions.

Consequently, the 0.50 threshold is retained only as a baseline experimental threshold, rather than being treated as the final system threshold.

11. Computational Design

11.1 Retrieval Computational Design

The retrieval stage is designed to reduce the computational workload of the downstream pairwise matching stage. The target datasets contain approximately **5.03 million S2 records** and **5.29 million S3 records**, making exhaustive pairwise comparison between every S1 record and every target record impractical.

Instead of performing all-to-all comparisons, the system constructs lookup indexes for the three retrieval views: normalized exact name, normalized exact address, and informative name tokens. Each S1 record is then used to perform direct lookups into these indexes, and the resulting candidates are combined through a union operation.

In the component-level validation experiments, the final retrieval configuration produced an average of **63.45 candidates per S1 for S2** and **64.09 candidates per S1 for S3**, compared with millions of possible target records per S1 under exhaustive comparison.

This substantially reduces the number of record pairs passed to the pairwise matching model while retaining a substantial portion of the known ground-truth matches.

The retrieval implementation is therefore used as a computational reduction layer before pairwise feature extraction and classification. The reported candidate counts are averages from the retrieval validation experiments and should not be interpreted as the final end-to-end system runtime.

12. Limitations / Failure Modes

12.1 Retrieval Limitations and Missed-Pair Analysis

The final three-view retrieval system does not achieve 100% recall. Therefore, retrieval establishes a recall ceiling for the downstream pairwise matching stage: if a true S1–S2 or S1–S3 pair is not retrieved, that pair cannot be evaluated by the pairwise model or considered by subsequent decision stages.

To understand this limitation, missed ground-truth pairs were analysed for the S2 retrieval experiment. Among **7,458 known true S2 matches**, **3,096 were not retrieved**, corresponding to the observed retrieval recall of **58.49%**.

The missed-pair analysis identified several recurring variation patterns. The categories are overlapping indicators and therefore are **not mutually exclusive**.

13. Reproducibility / Implementation Details

The T3 implementation is organized into separate feature engineering, model and training components.

Implementation Structure

```
src/  
— features/  
— pair_features.py  
— models/  
— pairwise_model.py  
  
scripts_train_pairwise.py  
  
models/  
└─ pairwise_model.pkl
```

Feature Engineering

`src/features/pair_features.py` contains the shared feature extraction pipeline, including:

- text normalization
- character similarity
- token similarity
- edit similarity
- numeric address extraction
- address-number overlap
- country agreement
- missing-value detection
- complete pairwise feature construction

The feature extractor accepts two records containing:

```
entity_id  
business_name
```

business_address
country

and returns the 12-feature representation used by the model.

Model Implementation

`src/models/pairwise_model.py` contains:

- feature-column definition
- feature-vector construction
- Logistic Regression training
- validation metric calculation
- model serialization/deserialization
- pairwise probability inference

The trained model is stored as:

`models/pairwise_model.pkl`

The saved model bundle includes both the trained classifier and the feature-column ordering required for reproducible inference.

Inference

For an S1 record and a retrieved S2/S3 candidate:

```
model, feature_columns = load_model(  
    "models/pairwise_model.pkl"  
)
```

```
p_match = predict_pair_probability(  
    model,  
    source1_record,  
    target_record  
)
```

The resulting probability is attached to the candidate relationship together with:

source1_entity_id
candidate_entity_id
candidate_source
target_source

match_probability

This scored candidate representation is then passed to the downstream graph/entity-level decision stage.

Versioning

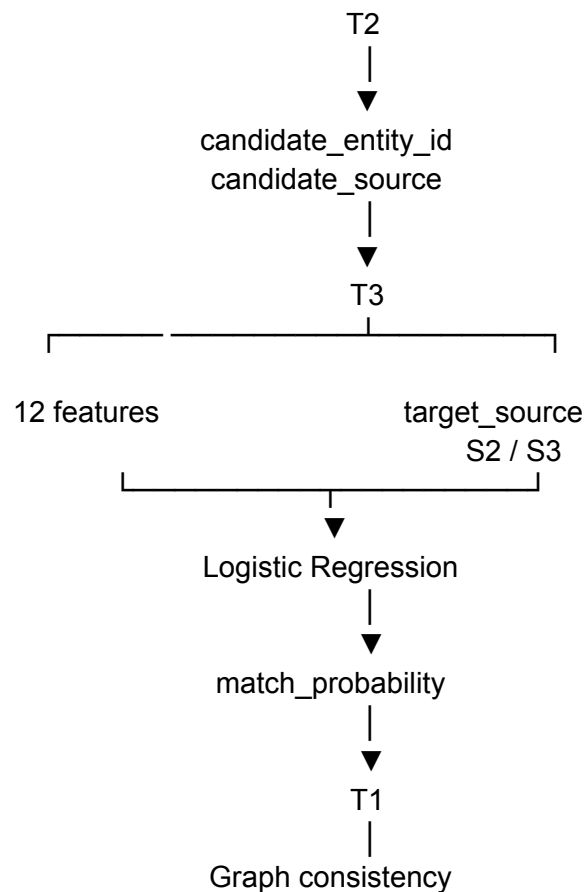
The implementation is committed under:

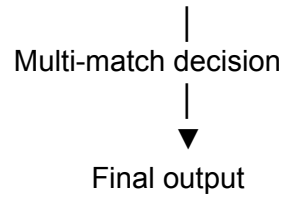
7f3c686 — Add pairwise matching model and features

and is available on the [t3-pairwise](#) branch.

T3 → T1 Interface Summary

For the final architecture, the cleanest handoff is:





This is the key distinction to preserve throughout the final documentation: T2 determines which candidates are worth scoring, T3 estimates how strongly each candidate pair matches, and T1 determines which relationships are jointly consistent at the entity level.